

MEMORIA TÉCNICA

SignaTour

*Plataforma web de itinerarios culturales adaptados
para personas sordas o con discapacidad auditiva*

Autora: Olatz González García

Proyecto Individual de Backend · Full Stack

Mayo de 2026

Repositorio: github.com/olatzglez/SignaTour

Índice

Índice.....	2
1. Introducción y Contexto.....	3
1.1 Antecedentes y Definición del Problema.....	3
1.2 Estado del Arte y Justificación.....	3
2. Análisis de Requisitos.....	4
2.1 Requisitos Funcionales (RF).....	4
2.2 Requisitos No Funcionales (RNF).....	5
3. Planificación y Gestión del Tiempo.....	6
3.1 Desglose de Tareas y Estimaciones (Backlog).....	6
3.2 Cronograma de Trabajo (Gantt).....	7
4. Análisis Funcional y Flujos.....	8
4.1 Diagrama de Flujo (User Flow).....	8
4.2 Interfaz de Usuario (UI).....	9
5. Arquitectura y Modelo de Datos.....	13
5.1 Diagrama de Clases.....	13
5.2 Diseño de la API (Endpoints Principales).....	15
5.3 Stack Tecnológico y Justificación.....	15
6. Retos Técnicos y Soluciones.....	16
6.1 Reto 1: Catálogo oficial de provincias bilingües.....	16
6.2 Reto 2: Doble interfaz (web SSR + API REST) compartiendo lógica.....	16
6.3 Reto 3: Accesibilidad WCAG AA desde el diseño.....	17
6.4 Reto 4: Reproducibilidad mediante Docker.....	18
6.5 Reto 5: Seguridad con JWT en cookie httpOnly.....	18
7. Pruebas de Calidad (QA).....	20
8. Conclusiones y Futuros Pasos.....	21
8.1 Aprendizajes principales.....	21
8.2 Mejoras para futuras versiones.....	21
8.3 Cierre.....	22

1. Introducción y Contexto

1.1 Antecedentes y Definición del Problema

En España viven más de un millón de personas con algún tipo de discapacidad auditiva. Para muchas de ellas, el acceso a la cultura, al patrimonio histórico y a las experiencias turísticas se ve limitado por barreras de comunicación que rara vez aparecen señalizadas de forma clara en las plataformas de información cultural más populares.

Aunque existen recursos de accesibilidad ya consolidados —Lengua de Signos Española (LSE), subtítulos, signoguías, bucle magnético y lectura fácil—, la información sobre qué museos, monumentos o rutas los ofrecen está fragmentada en webs institucionales, blogs especializados o folletos físicos. Una persona sorda que quiera planificar una salida cultural en una ciudad nueva puede invertir horas comprobando qué espacios garantizan condiciones de accesibilidad auditiva.

Las plataformas generalistas de itinerarios culturales y turismo activo, como Komoot, Tripadvisor o Google Maps, no ofrecen filtros específicos de accesibilidad auditiva. Y los pocos directorios temáticos que sí existen (Inclusite, Predif) están orientados a la consulta puntual, no a la planificación de rutas con múltiples paradas.

1.2 Estado del Arte y Justificación

Antes de iniciar el desarrollo se analizaron las soluciones existentes en el mercado:

- Komoot: excelente referente de UX para rutas con paradas numeradas, pero centrado en deporte al aire libre, sin información de accesibilidad para discapacidad auditiva.
- Inclusite: directorio español de turismo accesible muy completo, pero sin estructura de itinerarios encadenados ni filtros operativos.
- Predif: portal de referencia en accesibilidad turística, con un enfoque más institucional y menos centrado en la planificación visual.
- Plataformas turísticas generalistas (Tripadvisor, Visit Spain): no permiten filtrar por recursos de accesibilidad auditiva.

Itinerarios Accesibles nace para cubrir ese hueco. Es una plataforma vertical, centrada en un público específico, que combina lo mejor de los referentes analizados: la presentación visual y narrativa de Komoot (tarjetas con imagen y paradas tipo timeline) y la sensibilidad hacia los recursos accesibles de Inclusite (etiquetado por tipo, con iconografía clara). El proyecto prioriza la accesibilidad desde el diseño, cumpliendo el estándar WCAG 2.1 nivel AA, en coherencia con el público al que se dirige.

2. Análisis de Requisitos

Para garantizar la viabilidad del MVP en el plazo disponible se establecieron requisitos formales, separando los funcionales (qué debe hacer la aplicación) de los no funcionales (cómo debe comportarse).

2.1 Requisitos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Gestión de usuarios	El sistema diferencia entre los roles 'user' y 'admin'. Solo los admin pueden crear, modificar o eliminar itinerarios.
RF-02	Registro y autenticación	Los usuarios pueden registrarse con email y contraseña, e iniciar sesión recibiendo un token JWT que se guarda en la cookie httpOnly.
RF-03	Gestión de itinerarios	Los administradores pueden crear itinerarios indicando título, descripción, provincia, ciudad, duración y público objetivo, y modificarlos o eliminarlos posteriormente.
RF-04	Catálogo de recursos accesibles	Cada punto de interés puede asociarse a uno o varios recursos del catálogo (LSE, subtítulos, signoguía, bucle magnético, lectura fácil).
RF-05	Consulta pública	Cualquier visitante (autenticado o no) puede consultar el listado de itinerarios y el detalle de cada uno.
RF-06	Doble interfaz	La aplicación expone una interfaz web SSR para usuarios finales y una API REST JSON consumible desde un cliente externo.

2.2 Requisitos No Funcionales (RNF)

ID	Nombre	Descripción
RNF-01	Accesibilidad WCAG AA	Todas las pantallas deben cumplir el nivel AA del estándar WCAG 2.1: contraste mínimo 4.5:1, foco visible, etiquetado correcto y reflujo sin scroll horizontal.
RNF-02	Diseño responsive	Mobile-first con tres breakpoints (480, 768 y 1024 px). En móvil se sustituye la navegación principal por un menú hamburguesa lateral.
RNF-03	Seguridad	Las contraseñas se almacenan hasheadas con bcrypt. La autenticación se gestiona con JWT firmado, almacenado en la cookie httpOnly para mitigar ataques XSS.
RNF-04	Integridad referencial	La base de datos relacional define claves foráneas con políticas explícitas (CASCADE en borrado de itinerarios, SET NULL para usuarios borrados).
RNF-05	Reproducibilidad	El proyecto debe poder levantarse en cualquier máquina con Docker en menos de cinco minutos, mediante un docker-compose y un seed idempotente.
RNF-06	Mantenibilidad	El código se organiza en capas (controllers, services, models, middlewares) con responsabilidades separadas, facilitando futuras ampliaciones.

3. Planificación y Gestión del Tiempo

El proyecto se desarrolló entre el lunes 27 de abril y el lunes 4 de mayo de 2026, lo que supone un total de 6 días laborables (cinco días de la primera semana más el lunes final). Se estableció una capacidad efectiva de trabajo de entre 5 y 6 horas diarias dedicadas al código, reservando el resto del tiempo para documentación, pruebas manuales y resolución de bloqueos.

3.1 Desglose de Tareas y Estimaciones (Backlog)

Se priorizó cubrir todos los requisitos obligatorios del enunciado durante los primeros tres días, dejando los dos últimos para diseño visual, accesibilidad y documentación.

Fase	Tarea	Descripción técnica	Estimación
Diseño	Modelo y arquitectura	Diagrama entidad-relación, definición de las cinco tablas y de las relaciones 1:N y N:M, decisión sobre stack tecnológico.	0,5 días
Backend	Setup y modelos	Configuración de Express, Sequelize, PostgreSQL en Docker, definición de los modelos y sus asociaciones.	1 día
Backend	Autenticación y autorización	Endpoints de registro y login, hash con bcrypt, generación y verificación de JWT, middleware de roles.	1 día
Backend	CRUD itinerarios	Servicio compartido con validaciones, controladores API REST, errores propagados al handler central.	1 día
Frontend	Vistas SSR con PUG	Layout principal, header con dropdown, home, lista de itinerarios, detalle, login, registro y formulario admin.	1 día
Frontend	Diseño visual y responsive	Estilos por capas (styles, components, pages), paleta verde Komoot, tarjetas con imagen, menú hamburguesa móvil, WCAG AA.	1 día
Cierre	Pruebas y documentación	Pruebas manuales de los flujos críticos, redacción de la memoria técnica y limpieza del repositorio.	0,5 días

3.2 Cronograma de Trabajo (Gantt)

Por la corta duración del proyecto, las fases se solaparon parcialmente: las vistas SSR comenzaron en cuanto los modelos y la API estaban operativos, y el diseño visual se aplicó de forma incremental sobre las vistas ya existentes. La siguiente tabla resume la distribución temporal real del esfuerzo.

Fase / Día	Lun 27	Mar 28	Mié 29	Jue 30	Vie 1	Sáb-Do m	Lun 4
Modelo y setup	•	•					
Auth y CRUD API		•	•				
Vistas SSR (PUG)			•	•			
Provincias bilingües				•			
Diseño visual y CSS					•	•	
Responsive y WCAG						•	•
QA y memoria							•

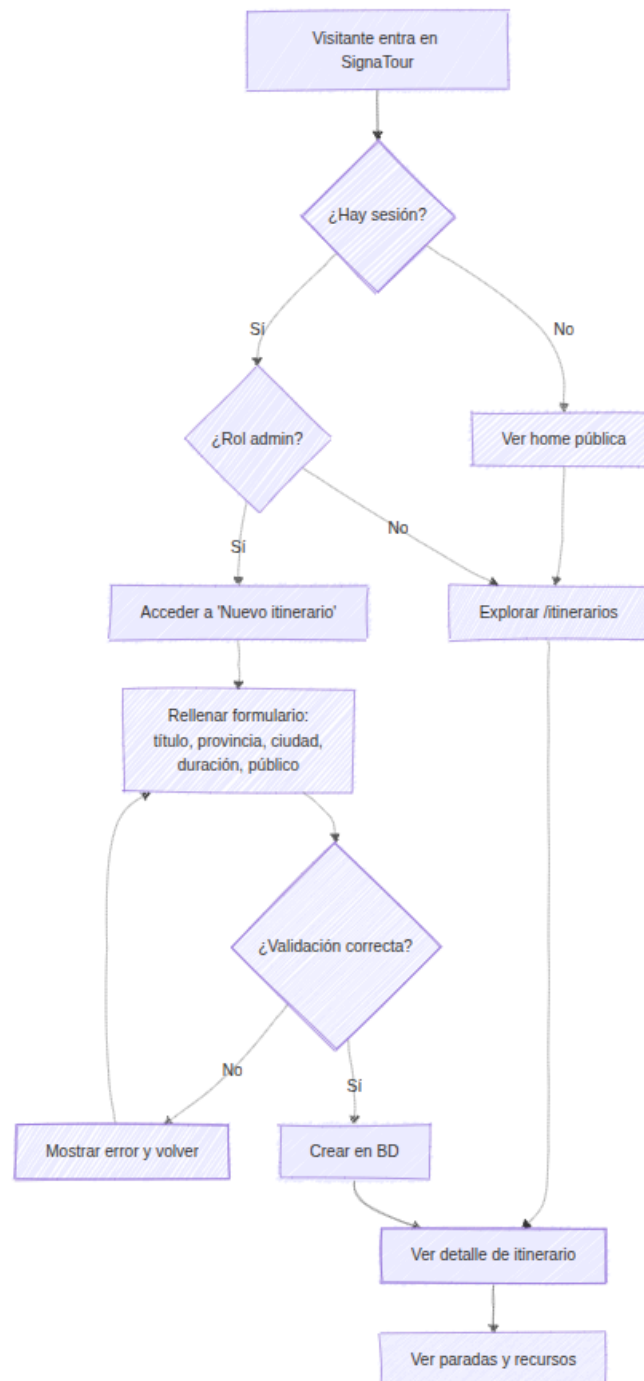
Los puntos verdes indican días de trabajo activo en cada fase. La fase de diseño visual y responsive se solapó con el fin de semana, en el que se realizaron tareas de menor complejidad técnica (descarga y procesamiento de imágenes, ajuste de paletas).

4. Análisis Funcional y Flujos

La aplicación contempla dos perfiles de usuario diferenciados: el visitante anónimo, que accede al catálogo público, y el administrador, que gestiona el contenido. A continuación se describe el flujo principal y se referencia la interfaz de usuario.

4.1 Diagrama de Flujo (User Flow)

El flujo principal puede representarse mediante el siguiente diagrama [Mermaid](#).



4.2 Interfaz de Usuario (UI)

La interfaz se inspira en dos referentes ya consolidados: Komoot, por su forma de presentar las rutas como tarjetas con imagen y sus paradas numeradas en línea de tiempo; y plataformas de turismo accesible como Inclusite, por la manera de etiquetar los recursos de accesibilidad mediante distintivos visuales con color e icono propios.

A continuación se referencian las pantallas principales en las capturas adjuntas:

- Home (/): hero con la propuesta de valor, sección de itinerarios destacados con tarjetas en grid responsive, sección de ciudades con imágenes y overlay degradado oscuro estilo Komoot.
- Lista de itinerarios (/itinerarios): grid de tarjetas (1 columna en móvil, 2 en tablet, 3 en escritorio), cada una con la imagen de la ciudad como cabecera y los badges de público y duración.
- Detalle de itinerario (/itinerarios/:id): cabecera con título, ciudad y descripción destacada, seguido de un timeline vertical numerado con cada parada y los recursos de accesibilidad asociados como pills coloreadas con iconos.
- Formulario de creación (/itinerarios/nuevo): solo accesible para administradores, con un select cerrado de las 52 provincias en denominación oficial bilingüe, campos obligatorios marcados, validación cliente y servidor.
- Login y registro (/login, /registro): formularios sobrios con feedback de error claro y enlaces cruzados entre las dos vistas.

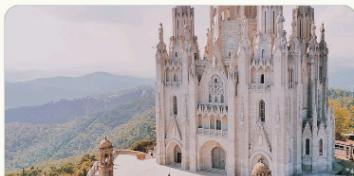
Cultura accesible para todas las personas

Descubre itinerarios culturales adaptados con lengua de signos, subtítulos, signoguías, bucle magnético y lectura fácil.

Ver itinerarios

Itinerarios destacados

Selección de rutas accesibles para descubrir.



Granada y la Alhambra

Granada, Granada

Visita al conjunto monumental nazarí y al barrio del Albaicín con recursos accesibles.

Todos 180 min

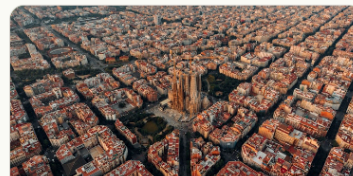


Donostia gastronómica

Donostia / San Sebastián, Gipuzkoa

Ruta de pintxos por la Parte Vieja con bucle magnético y subtítulos.

Adultos 120 min



Barcelona modernista accesible

Barcelona, Barcelona

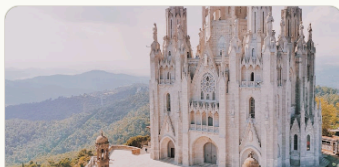
Recorrido por las obras maestras de Gaudí con recursos para personas sordas.

+12 150 min

Home

Itinerarios accesibles

Explora rutas culturales adaptadas con recursos de accesibilidad.



Granada y la Alhambra

Granada, Granada

Visita al conjunto monumental nazarí y al barrio del Albaicín con recursos accesibles.

Todos 180 min

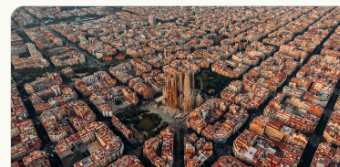


Donostia gastronómica

Donostia / San Sebastián, Gipuzkoa

Ruta de pintxos por la Parte Vieja con bucle magnético y subtítulos.

Adultos 120 min



Barcelona modernista accesible

Barcelona, Barcelona

Recorrido por las obras maestras de Gaudí con recursos para personas sordas.

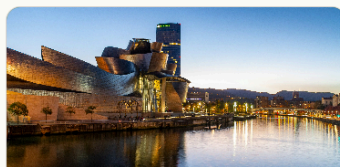
+12 150 min



Sevilla nocturna para adultos

Sevilla, Sevilla

Ruta nocturna por el casco antiguo con tapas y flamenco accesible.



Bilbao y el Guggenheim

Bilbao, Bizkaia

Ruta por el centro de Bilbao con visita accesible al museo.



Madrid de los Austrias accesible

Madrid, Madrid

Recorrido por el Madrid histórico con paradas accesibles para personas sordas.

Lista de itinerarios

ItinerariosCiudadesRecursos accesiblesU

Recorrido

3 paradas

1

Alhambra - Palacios Nazaríes
C/ Real de la Alhambra, s/n, 18009 Granada
Signoguías en LSE y audioguías con subtítulos.
Accesibilidad:

Signoguía

Lengua de Signos Española

Subtítulos

2

Mirador de San Nicolás
Pl. Mirador de San Nicolás, 18010 Granada
Mirador con paneles informativos en lectura fácil sobre la vista a la Alhambra.
Accesibilidad:

Lectura Fácil

3

Catedral de Granada
C/ Gran Vía de Colón, 5, 18001 Granada
Visitas con bucle magnético en la sala principal.
Accesibilidad:

Bucle Magnético

Signoguía

Detalle de un itinerario

U

Nuevo itinerario

Rellena los datos para crear un itinerario accesible.

Título *

Provincia *

Selecciona una provincia

Ciudad *

Descripción

Duración (minutos) *

Formulario nuevo itinerario (admin).

Usuaría[Mi perfil](#)[Itinerarios](#)[Ciudades](#)[Nuevo itinerario](#)[Gestionar recursos](#)[Cerrar sesión](#)**Itinerarios destacados**

Selección de rutas accesibles para descubrir.



Vista móvil con menú responsive abierto (admin).

5. Arquitectura y Modelo de Datos

5.1 Diagrama de Clases

El modelo de datos se compone de cinco entidades, con dos tipos de relaciones: una jerárquica (1:N) entre Itinerario y PuntoInteres, y una asociativa (N:M) entre PuntoInteres y RecursoAccesibilidad mediante una tabla puente. Adicionalmente, User mantiene una relación 1:N opcional con Itinerario, registrando qué administrador creó cada uno.

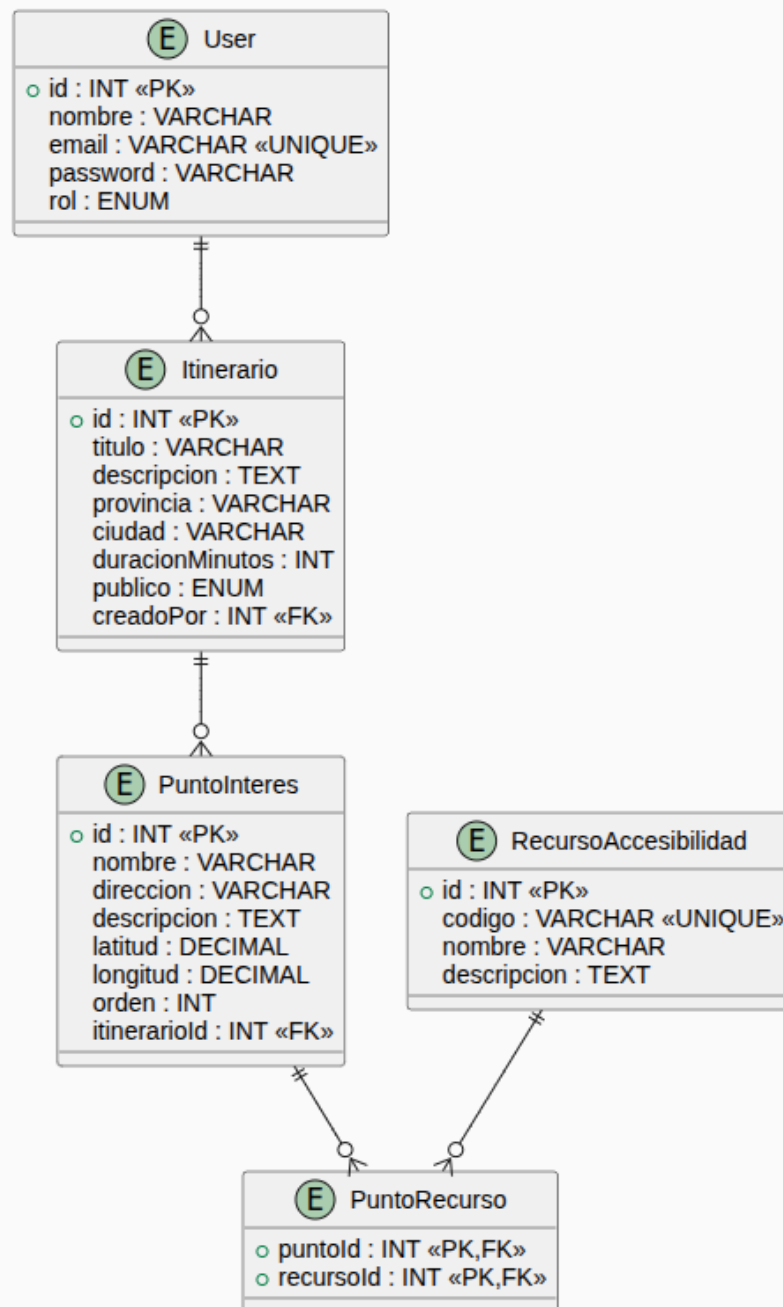


Diagrama de clases hecho con PlantUML

Las entidades, sus atributos principales y sus relaciones se detallan en la siguiente tabla descriptiva, que sirve como apoyo textual al diagrama:

Entidad	Atributos clave	Relaciones
User	id (PK), nombre, email (UNIQUE), password (bcrypt), rol ENUM('user','admin')	1:N con Itinerario (creadoPor, ON DELETE SET NULL)
Itinerario	id (PK), titulo, descripcion, provincia (NOT NULL), ciudad, duracionMinutos, publico ENUM, creadoPor (FK)	1:N con PuntoInteres (ON DELETE CASCADE), N:1 con User
PuntoInteres	id (PK), nombre, direccion, descripcion, latitud DECIMAL(10,7), longitud DECIMAL(10,7), orden, itinerarioid (FK)	N:1 con Itinerario, N:M con RecursoAccesibilidad
RecursoAccesibilidad	id (PK), codigo (UNIQUE), nombre, descripcion. Catálogo: LSE, SUBTITULOS, SIGNOGUIA, BUCLE_MAGNETICO, LECTURA_FACIL	N:M con PuntoInteres (a través de PuntoRecurso)
PuntoRecurso	puntoid (FK), recursoid (FK). PK compuesta. Tabla puente.	ON DELETE CASCADE en ambos lados

5.2 Diseño de la API (Endpoints Principales)

La API sigue las convenciones REST con nomenclatura de recursos en plural y verbos HTTP semánticos. Todos los endpoints de escritura están protegidos con `verifyToken`; los de gestión, además, con `requireRole('admin')`.

Método	Endpoint	Descripción	Auth
POST	/api/auth/register	Registro de nuevo usuario con hash de contraseña.	No
POST	/api/auth/login	Autenticación, devuelve JWT en cookie y JSON.	No
GET	/api/auth/me	Datos del usuario autenticado.	Sí
GET	/api/itinerarios	Lista pública de itinerarios.	No
GET	/api/itinerarios/:id	Detalle público con puntos y recursos anidados.	No
POST	/api/itinerarios	Crear itinerario (solo administrador).	Admin
PATCH	/api/itinerarios/:id	Actualizar itinerario parcialmente.	Admin
DELETE	/api/itinerarios/:id	Eliminar itinerario (cascada en puntos y recursos).	Admin

5.3 Stack Tecnológico y Justificación

- **Backend:** Node.js con Express. Elegido por su arquitectura no bloqueante, ecosistema maduro y por ser el stack en el que se ha trabajado durante el bootcamp.
- **Base de datos:** PostgreSQL 16, ejecutado en contenedor Docker. Se eligió frente a MongoDB por la naturaleza relacional de los datos (las relaciones N:M entre puntos y recursos se modelan de forma natural con SQL).
- **ORM:** Sequelize 6. Permite definir modelos en JavaScript, sincronizar el esquema, gestionar asociaciones y aprovechar las validaciones a nivel de modelo, además de generar SQL portable.
- **Vistas:** PUG con renderizado en servidor (SSR). Frente a la opción de una SPA con React, el SSR garantiza HTML semántico de partida, mejor accesibilidad para lectores de pantalla y carga inicial más rápida.
- **Autenticación:** JWT firmado con secreto, almacenado en cookie `httpOnly` (mitigación XSS) y duplicado en cabecera `Authorization` para clientes API. Hash de contraseñas con `bcrypt`.
- **Contenedores:** Docker Compose orquesta Postgres y pgAdmin, garantizando que cualquier persona pueda levantar el proyecto sin instalar nada localmente.

6. Retos Técnicos y Soluciones

A continuación se describen los cinco retos técnicos más relevantes del proyecto, con la situación de partida, la investigación realizada y la solución implementada.

6.1 Reto 1: Catálogo oficial de provincias bilingües

Inicialmente el modelo Itinerario solo contemplaba un campo libre 'ciudad'. Al revisar la coherencia de los datos se detectó la conveniencia de añadir 'provincia' como campo administrativo. La pregunta fue cómo presentarlas: ¿en castellano? ¿en su lengua cooficial? ¿con barra?

Tras consultar la legislación vigente (leyes 19/2011, 2/1998, 2/1992 y 13/1997 sobre denominación oficial de territorios), se optó por usar la denominación oficial cooficial donde está reconocida (A Coruña, Bizkaia, Gipuzkoa, Girona, Illes Balears, Lleida y Ourense), y el castellano para el resto. La lista se aisló en su propio módulo, permitiendo reutilizarla desde cualquier punto:

```
// src/constants/provincias.js
export const PROVINCIAS_ESPANA = [
  'A Coruña', 'Álava', 'Albacete', 'Alicante', 'Almería',
  'Asturias', 'Ávila', 'Badajoz', 'Barcelona', 'Bizkaia',
  'Burgos', 'Cáceres', 'Cádiz', 'Cantabria', /* ... */
  'Gipuzkoa', 'Girona', /* ... */ 'Lleida', /* ... */
  'Ourense', /* ... */ 'Zaragoza',
]
```

El select del formulario admin se alimenta de este array, garantizando coherencia entre el dato administrativo y la denominación oficial.

6.2 Reto 2: Doble interfaz (web SSR + API REST) compartiendo lógica

El enunciado pedía explícitamente una API REST. Sin embargo, el proyecto buscaba también un cliente web accesible y completo. La pregunta fue: ¿se duplica la lógica de negocio en dos backends, o se encuentra una forma de compartirla?

La solución fue introducir una capa de servicios (src/services/) consumida por dos conjuntos de controladores: uno bajo src/controllers/api/ que devuelve JSON, y otro bajo src/controllers/web/ que llama a res.render(). Ambos delegan toda la lógica en el mismo servicio:

```
// src/services/itinerario.service.js
const create = async (datos, creadoPor) => {
  if (!datos.titulo || !datos.provincia || !datos.ciudad) {
    const error = new Error('Faltan campos obligatorios')
    error.statusCode = 400
    throw error
  }
  return await Itinerario.create({ ...datos, creadoPor })
}

// src/controllers/api/itinerario.controller.js
const create = async (req, res, next) => {
  try {
    const it = await itinerarioService.create(req.body, req.usuario.id)
    res.status(201).json(it)
  } catch (e) { next(e) }
}
```



```
// src/controllers/web/itinerario.controller.js
const procesarFormulario = async (req, res) => {
  try {
    const it = await itinerarioService.create(req.body, req.usuario.id)
    res.redirect(`/itinerarios/${it.id}`)
  } catch (e) {
    res.render('itinerarios/formulario', { error: e.message, valores: req.body })
  }
}
```

Con este patrón, una validación o cambio de regla se aplica de forma simultánea a la API y a la web. Los errores se propagan con `statusCode` al middleware central, que decide si responder JSON o renderizar una vista.

6.3 Reto 3: Accesibilidad WCAG AA desde el diseño

Dado que el público objetivo son personas sordas o con discapacidad auditiva, la accesibilidad no podía ser un añadido posterior, sino un criterio rector desde el primer commit del CSS.

Se utilizó el WebAIM Contrast Checker para verificar que todos los pares texto-fondo cumplieran el ratio mínimo de 4.5:1. La paleta verde inspirada en Komoot se ajustó hasta cumplir AA en todas sus combinaciones. Las variables CSS centralizadas en `:root` facilitaron la auditoría:

```
/* public/css/styles.css */
:root {
  --color-primario: #2d6a4f;          /* AA sobre blanco: 6.7:1 */
  --color-primario-oscuro: #1b4332;   /* AA sobre blanco: 11.5:1 */
  --color-texto: #1a1a1a;             /* AA sobre fondo: 16:1 */
  --color-acento: #e85d2a;            /* Foco visible 3px */
  --tamano-toque-min: 44px;           /* Criterio AAA */
}

/* Foco visible reforzado */
a:focus-visible, button:focus-visible {
  outline: 3px solid var(--color-acento);
  outline-offset: 3px;
}

/* Respeto a prefers-reduced-motion */
@media (prefers-reduced-motion: reduce) {
  *, ::before, ::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}
```

Adicionalmente, todos los inputs se etiquetaron con `for/id`, el menú hamburguesa móvil utiliza atributos ARIA (`aria-expanded`, `aria-hidden`, `aria-controls`), se incluyó un skip-link y se garantizó que toda la interfaz fuera navegable con teclado.

6.4 Reto 4: Reproducibilidad mediante Docker

Para que cualquier persona pudiera levantar el proyecto sin instalar PostgreSQL ni configurar usuarios, se optó por contenerizar la base de datos. El `docker-compose.yml` define dos servicios: `postgres` (en el puerto 5434:5432 para evitar colisiones con instalaciones locales) y `pgadmin` (en el 5050).

```
# docker-compose.yml
services:
  postgres:
    image: postgres:16
    ports: ["5434:5432"]
    environment:
      POSTGRES_USER: itinerarios
      POSTGRES_PASSWORD: itinerarios_pass
      POSTGRES_DB: itinerarios
    volumes:
      - postgres_data:/var/lib/postgresql/data

  pgadmin:
    image: dpage/pgadmin4
    ports: ["5050:80"]
```

Combinado con un seed idempotente (`findOrCreate` en lugar de `create`), la puesta en marcha se reduce a tres comandos: `docker compose up -d`, `npm run dev` y `npm run seed`. El volumen `postgres_data` garantiza que los datos persistan entre reinicios del contenedor.

6.5 Reto 5: Seguridad con JWT en cookie `httpOnly`

La pregunta fue cómo gestionar la sesión del usuario tanto en la web SSR como en la API REST sin duplicar mecanismos. La solución más limpia fue usar JWT en una cookie `httpOnly` (que el navegador envía automáticamente con cada petición y que JavaScript del cliente no puede leer, mitigando ataques XSS), aceptando además el token en la cabecera `Authorization` para clientes API que no usen cookies.

```
// src/controllers/api/auth.controller.js
const login = async (req, res, next) => {
  try {
    const { token, usuario } = await authService.login(req.body)
    res.cookie('token', token, {
      httpOnly: true,
      sameSite: 'lax',
      maxAge: 24 * 60 * 60 * 1000 // 24 horas
    })
    res.json({ usuario })
  } catch (e) { next(e) }
}

// src/middlewares/auth.middleware.js
export const verifyToken = (req, res, next) => {
  const token = req.cookies.token ||
    req.headers.authorization?.replace('Bearer ', '')
  if (!token) return next({ statusCode: 401, message: 'Auth requerida' })
  try {
    req.usuario = jwt.verify(token, process.env.JWT_SECRET)
    next()
  } catch { next({ statusCode: 401, message: 'Token inválido' }) }
}

// Factory de middlewares para roles
```

```
export const requireRole = (rol) => (req, res, next) => {  
  if (req.usuario?.rol !== rol) return next({ statusCode: 403, message: 'Sin  
permisos' })  
  next()  
}
```

El middleware `verifyToken` se reutiliza en ambos sets de rutas. Para los endpoints que requieren rol específico se compone con `requireRole('admin')`, aplicando el principio de responsabilidad única.

7. Pruebas de Calidad (QA)

Antes del cierre del proyecto se ejecutó una batería de pruebas manuales sobre los flujos críticos (happy path) y sobre los casos límite previsible (edge cases). En esta versión del proyecto no se han implementado pruebas automatizadas; se reconocen como deuda técnica para futuras iteraciones.

Caso de prueba	Resultado esperado	Estado
Login con credenciales correctas	El servidor responde 200, devuelve los datos del usuario y crea la cookie token httpOnly. Tras el login, el avatar muestra la inicial.	✓ Pasado
Login con contraseña incorrecta	Devuelve 401 con un mensaje genérico ('Credenciales inválidas') sin revelar si el email existe.	✓ Pasado
Crear itinerario sin sesión	Al intentar acceder a /itinerarios/nuevo sin estar logueado, redirige a /login.	✓ Pasado
Crear itinerario sin rol admin	Un usuario con rol 'user' que intente acceder a la creación recibe un 403 ('Sin permisos').	✓ Pasado
Validación de formulario	Enviar el formulario de creación sin provincia, ciudad o duración devuelve un 400 y vuelve a renderizar el formulario con los valores ya escritos y el mensaje de error.	✓ Pasado
Eliminación en cascada	Al borrar un itinerario, sus puntos de interés y las relaciones N:M en PuntoRecurso se eliminan automáticamente (ON DELETE CASCADE).	✓ Pasado
Navegación con teclado	Recorrer toda la home con Tab muestra siempre un foco visible. Escape cierra el menú lateral móvil. Enter activa los botones.	✓ Pasado
Reflujo en móvil	A 320px de ancho, ningún elemento provoca scroll horizontal. El menú hamburguesa abre y cierra correctamente.	✓ Pasado
Contraste de color	Verificación con WebAIM Contrast Checker de los pares texto/fondo principales. Todos cumplen ratio $\geq 4.5:1$.	✓ Pasado

8. Conclusiones y Futuros Pasos

El desarrollo de SignaTour ha permitido consolidar conocimientos sobre arquitectura backend, modelado relacional, autenticación basada en tokens y diseño accesible. El sistema es funcional y cumple con los requisitos académicos del proyecto, manteniendo un equilibrio entre simplicidad y solidez técnica, en línea con el principio de 'menos es más, si está bien hecho'.

8.1 Aprendizajes principales

- La elección de SSR frente a SPA tiene un impacto directo en accesibilidad: el HTML semántico generado en servidor es leído correctamente por lectores de pantalla desde el primer byte.
- Una capa de servicios bien diseñada permite que la misma lógica de negocio dé respuesta a varias interfaces (web y API) sin duplicación, reduciendo el riesgo de divergencia.
- La accesibilidad WCAG AA como criterio de diseño desde el inicio es mucho más eficiente que aplicarla como capa final: muchas decisiones (variables CSS, focus-visible, target size) son baratas si se toman pronto y caras si se aplazan.
- El uso de Docker para la base de datos elimina por completo la fricción de la configuración inicial y garantiza que el proyecto sea reproducible en cualquier máquina.
- La denominación oficial bilingüe de las provincias, aunque sea un detalle, refleja el espíritu inclusivo del proyecto y demuestra que el código puede ser una herramienta de respeto cultural.

8.2 Mejoras para futuras versiones

1. Sistema de filtrado y paginación en GET /api/itinerarios: filtros por provincia, público y recurso de accesibilidad; paginación basada en offset/limit.
2. Documentación automática de la API con Swagger / OpenAPI 3, generada a partir de comentarios JSDoc en los controladores.
3. Mapa interactivo en el detalle de itinerario, mostrando los puntos con sus coordenadas reales mediante OpenStreetMap (Leaflet).
4. Subida de imágenes propias por itinerario, permitiendo al administrador adjuntar fotografías sin depender de las imágenes asociadas a la ciudad.
5. Pruebas automatizadas con Jest y Supertest para los servicios y los endpoints de la API.
6. Internacionalización (i18n) con traducción a euskera, catalán y gallego, en coherencia con la decisión de las provincias bilingües.
7. Sistema de notificaciones que avise a los administradores cuando se publique un nuevo recurso de accesibilidad reconocido oficialmente.

8.3 Cierre

Más allá del cumplimiento técnico, este proyecto ha sido una oportunidad para reflexionar sobre el papel del software como herramienta de inclusión. Una plataforma cultural accesible no es un producto técnicamente diferente de otra cualquiera: es solamente un producto técnicamente bien hecho, con el público adecuado en mente desde el primer momento.

Repositorio: github.com/olatzglez/SignaTour